

Log Analyzer Agent — Complete Walkthrough Guide

Purpose: End-to-end guide for understanding, running, and extending the Log Analyzer Agent.

No LLM API required — this is a pure skills-based local agent.

Table of Contents

1. [How to Execute the Main Agent](#)
 2. [What Each Skill Does and How It Gets Invoked](#)
 3. [Where to Run: VS Code vs UI Options](#)
 4. [High-Level Architecture Diagram](#)
 5. [Low-Level Component Diagram](#)
 6. [How to Generate and View the Report](#)
-

1. How to Execute the Main Agent

Prerequisites

```
bash

# Navigate to project root
cd log-analyzer-agent

# Install dependencies (no API key needed)
pip install -r requirements.txt --break-system-packages
```

`requirements.txt` only needs:

```
anthropic>=0.18.0    # imported but API key is optional for local mode
pyyaml>=6.0           # for parsing SKILL.md frontmatter
```

Execution Modes

Mode A — Direct File Analysis (Most Common)

```
bash

python agent/main_agent.py --file <path-to-log-file>
```

Parameter	Short	Required	Description
<code>--file</code>	<code>-f</code>	Yes (unless interactive)	Path to the log file to analyze
<code>--interactive</code>	<code>-i</code>	No	Start an interactive chat session

Examples:

```
bash
```

```
# Analyze a specific log file
python agent/main_agent.py --file /var/log/nginx/access.log

# Analyze sample Apache log (included in project)
python agent/main_agent.py --file tests/sample_data/apache_access.log

# Analyze JSON logs
python agent/main_agent.py --file tests/sample_data/json_logs.log

# Analyze application logs
python agent/main_agent.py --file tests/sample_data/application.log
```

What happens internally when you run this:

1. LogAnalyzerAgent.__init__()
 - └ SkillLoader loads all 4 SKILL.md files
 - └ Prints: "✓ Loaded skill: log-parsing" × 4
2. execute_skill_workflow("analyze this log file: path", log_file_path)
 - └ determine_required_skills() ← keyword match on query
 - └ Returns: ['log-parsing', 'error-detection', 'pattern-analysis', 'report-generation']
3. For each skill in order:
 - └ log-parsing → LogParser.parse_file() → stores parsed_logs
 - └ error-detection → ErrorDetector.detect_errors() → stores errors
 - └ pattern-analysis → PatternAnalyzer.analyze_patterns() → stores patterns
 - └ report-generation → ReportGenerator.generate_report() → saves to outputs/

Console output you'll see:

=====
Log Analyzer Agent - Initializing

=====
Loading skills...

- ✓ Loaded skill: log-parsing
- ✓ Loaded skill: error-detection
- ✓ Loaded skill: pattern-analysis
- ✓ Loaded skill: report-generation
- ✓ Loaded 4 skills

=====

📋 Required skills: log-parsing, error-detection, pattern-analysis, report-generation

▶ Executing: log-parsing

Detected format: apache

- ✓ Parsed 20 log entries

▶ Executing: error-detection

- ✓ Found 4 errors (3 unique types)

▶ Executing: pattern-analysis

- ✓ Identified 2 patterns, 0 anomalies

▶ Executing: report-generation

- ✓ Report saved to: outputs/log_analysis_report_20260521_143052.md

Analysis complete! Report saved to:
outputs/log_analysis_report_20260521_143052.md

Executive Summary

Status:  GOOD

...

Mode B — Interactive Session

```
bash

python agent/main_agent.py --interactive
# or simply (defaults to interactive when no --file given):
python agent/main_agent.py
```

Available commands inside interactive mode:

```
You: analyze tests/sample_data/application.log
You: find errors in /var/log/app.log
You: show patterns in /tmp/server.log
You: generate report for /tmp/server.log
You: clear          ← resets conversation state
You: quit           ← exits
```

The agent uses **keyword matching** to determine which skills to activate:

If your query contains...	Skills activated
<code>.log</code> , <code>parse</code> , <code>read log</code>	log-parsing
<code>error</code> , <code>failure</code> , <code>exception</code> , <code>critical</code> , <code>detect</code>	log-parsing + error-detection
<code>pattern</code> , <code>anomaly</code> , <code>trend</code> , <code>spike</code> , <code>unusual</code>	log-parsing + pattern-analysis
<code>report</code> , <code>summary</code> , <code>export</code> , <code>generate report</code>	all 4 skills
(log file path with no keyword)	all 4 skills

Mode C — Individual Tools (CLI)

You can run each tool independently for custom workflows:

```
bash
```

```
# Step 1: Parse only
python tools/log_parser.py tests/sample_data/apache_access.log
# → Saves: tests/sample_data/apache_access.log_parsed.json

# Step 2: Detect errors from parsed output
python tools/error_detector.py tests/sample_data/apache_access.log_parsed.json
# → Saves: apache_access.log_parsed_errors.json

# Step 3: Analyze patterns from parsed output
python tools/pattern_analyzer.py tests/sample_data/apache_access.log_parsed.json
# → Saves: apache_access.log_parsed_patterns.json

# Step 4: Extract specific fields
python skills/log-parsing/scripts/extract_fields.py \
    --input parsed.json \
    --fields timestamp,level,message \
    --output fields.json

# Step 5: Normalize timestamps only
python skills/log-parsing/scripts/normalize_timestamps.py \
    --input parsed.json \
    --output normalized.json

# Step 6: Categorize errors only
python skills/error-detection/scripts/categorize_errors.py \
    --input parsed.json \
    --output categorized.json

# Step 7: Find patterns only
python skills/pattern-analysis/scripts/find_patterns.py \
    --input parsed.json \
```



```
--min-freq 3 \  
--output patterns.json
```

2. What Each Skill Does and How It Gets Invoked

The agent follows the **Claude Skills Handbook** pattern. Each skill is a folder with:

- `SKILL.md` — YAML frontmatter (metadata + trigger keywords) + detailed instructions
- `scripts/` — standalone Python helper scripts
- `references/` — detailed documentation loaded on-demand

How Skills Get Loaded (Progressive Disclosure)

Level 1 — Always loaded (frontmatter only, ~5 lines):

name, description, trigger keywords, compatibility

→ Used by agent to decide: "do I need this skill?"

Level 2 — Loaded when skill is ACTIVE (full instructions):

Step-by-step execution instructions, error handling, examples

→ Injected into system prompt only for required skills

Level 3 — On-demand (references/ folder):

log-formats.md, error-patterns.md, pattern-types.md, report-templates.md

→ Referenced by skills when detailed specs are needed

Skill 1: `log-parsing`

Trigger keywords: `parse`, `read log`, `.log`, `analyze log`, `log file`

What it does:

1. Detects log format automatically (Apache, Nginx, syslog, JSON, Docker, K8s, application, custom)
2. Parses each line into structured Python dict with normalized fields
3. Extracts: timestamp, level, ip_address, method, path, status_code, message, raw_line
4. Normalizes all timestamps to ISO 8601 UTC format
5. Detects and combines multiline entries (stack traces)
6. Processes large files in 10,000-line chunks
7. Reports parse statistics (total, parsed, errors, success rate)

Invoked by:

```
python

# Inside execute_skill_workflow() in main_agent.py:
parser = LogParser()
parsed_logs = parser.parse_file(log_file_path)
# Returns: { format, total_lines, parsed_lines, errors, entries: [...], meta...
```

Helper scripts (run independently or called by tools):

- `scripts/extract_fields.py` — extract specific fields from parsed JSON
- `scripts/normalize_timestamps.py` — convert all timestamps to ISO 8601

Reference: `references/log-formats.md` — complete regex patterns and field definitions for all 8 formats

Skill 2: `error-detection`

Trigger keywords: `error`, `failure`, `exception`, `critical`, `find error`, `detect`, `what failed`

Depends on: log-parsing (must run first)

What it does:

- 1. Filters entries by log level (CRITICAL, ERROR, FATAL) or HTTP 5xx status codes
- 2. Categorizes each error into one of 8 categories:

Category	Keywords detected
database	connection, query, sql, timeout, deadlock
network	refused, socket, dns, unreachable
authentication	auth, unauthorized, forbidden, token
application	nullpointer, exception, runtime
configuration	config, missing, not found
resource	memory, disk, out of, exhausted
timeout	timeout, timed out, deadline
validation	invalid, constraint, format

3. Groups similar errors using message parameterization (replaces numbers/IPs/UUIDs with placeholders)
4. Calculates metrics: error rate, error % of total, errors per hour, distribution by category
5. Identifies critical errors (CRITICAL level, frequency > 100, or known critical keywords)
6. Creates hourly error timeline for trend visualization

Invoked by:

```
python

detector = ErrorDetector()
errors = detector.detect_errors(parsed_logs)
# Returns: { total_errors, unique_error_types, error_percentage,
#           grouped_errors, critical_error_list, timeline, top_errors }
```

Helper script: `scripts/categorize_errors.py` — standalone categorization

Reference: `references/error-patterns.md` — catalog of error patterns, stack trace processing, severity matrix

Skill 3: `pattern-analysis`

Trigger keywords: `pattern`, `anomaly`, `trend`, `unusual`, `spike`, `correlat`, `behavior`

Depends on: log-parsing (must run first)

What it does:

1. **Temporal patterns** — finds errors recurring at regular intervals (e.g., every 30 min = scheduled job issue)

2. **Request patterns** — finds endpoints called with unusual frequency
3. **Error message patterns** — groups repeated error messages
4. **Volume anomalies** — flags hours where traffic is > 3 standard deviations from mean
5. **Response time anomalies** — detects slow requests (if response_time field present)
6. **Error rate anomalies** — flags sudden spikes in error rate
7. **Trend analysis** — calculates if traffic/error rate is increasing, decreasing, or stable
8. **Correlation** — links high traffic periods to error spikes
9. **Peak times** — identifies busiest and quietest hours
10. **Insights** — generates actionable human-readable recommendations from patterns

Invoked by:

```
python

analyzer = PatternAnalyzer()
patterns = analyzer.analyze_patterns(parsed_logs)
# Returns: { pattern_count, anomaly_count, patterns, anomalies,
#           trends, correlations, peak_times, insights }
```

Helper script: `scripts/find_patterns.py` — standalone pattern finder

Reference: `references/pattern-types.md` — pattern classification, confidence calculation, real-world examples

Skill 4: `report-generation`

Trigger keywords: `report`, `summary`, `export`, `generate report`, `show result`

Depends on: all previous skills (aggregates their outputs)

What it does:

- 1. Aggregates results from log-parsing, error-detection, pattern-analysis
- 2. Determines overall status:  GOOD /  WARNING /  CRITICAL
- 3. Creates executive summary with top findings
- 4. Generates detailed sections (parsing stats, error analysis, pattern analysis)
- 5. Creates ASCII visualizations (error timeline bar chart, distribution chart)
- 6. Generates prioritized recommendations with effort estimates
- 7. Saves report in requested format to `outputs/` directory

Invoked by:

```
python

generator = ReportGenerator()
report = generator.generate_report(results)    # results = all skill outputs
output_path = generator.save_report(report, format="markdown")
```

Output formats:

Format	File extension	Best for
Markdown	<code>.md</code>	Developers, documentation, git
HTML	<code>.html</code>	Management, email, browser view
JSON	<code>.json</code>	Dashboards, APIs, data pipelines
Text	<code>.txt</code>	Terminal, plain email, logging

Reference: `references/report-templates.md` — complete templates for all formats and section layouts

3. Where to Run: VS Code vs UI Options

Option A — VS Code Terminal (Primary Method)

```
bash

# Open integrated terminal (Ctrl+`)
cd log-analyzer-agent
python agent/main_agent.py --file tests/sample_data/application.log
```

VS Code tips:

- Install the **Python extension** for syntax highlighting
 - Install **Pylance** for autocomplete
 - Open `agent/main_agent.py` to see the agent code
 - Open `skills/log-parsing/SKILL.md` to read skill instructions
 - The `outputs/` folder auto-appears in the Explorer after first run
-

Option B — VS Code + Python Interactive Window

Create a notebook-style experience inside VS Code:

```
python
```

```
# In any .py file, add # %% to create cells, then run with Shift+Enter
# %%
import sys
sys.path.insert(0, '/path/to/log-analyzer-agent')

from tools.log_parser import LogParser
from tools.error_detector import ErrorDetector
from tools.report_generator import ReportGenerator

# %%
parser = LogParser()
parsed = parser.parse_file('tests/sample_data/application.log')
print(f"Parsed {parsed['parsed_lines']} entries, format: {parsed['format']}")

# %%
from tools.error_detector import ErrorDetector
detector = ErrorDetector()
errors = detector.detect_errors(parsed)
print(f"Found {errors['total_errors']} errors")
```

Option C — Gradio Web UI (Browser Interface)

Create `ui/gradio_app.py`:

```
python
```



```
import gradio as gr
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

from tools.log_parser import LogParser
from tools.error_detector import ErrorDetector
from tools.pattern_analyzer import PatternAnalyzer
from tools.report_generator import ReportGenerator

def analyze_log_file(file_obj, report_format):
    """Analyze uploaded log file and return report"""
    if file_obj is None:
        return "Please upload a log file.", ""

    log_file_path = file_obj.name

    # Run analysis pipeline
    parser = LogParser()
    parsed = parser.parse_file(log_file_path)

    detector = ErrorDetector()
    errors = detector.detect_errors(parsed)

    analyzer = PatternAnalyzer()
    patterns = analyzer.analyze_patterns(parsed)

    # Generate report
    generator = ReportGenerator()
    results = {
        'log-parsing': parsed,
        'error-detection': errors,
        'pattern-analysis': patterns
```

```

}
report = generator.generate_report(results)
output_path = generator.save_report(report, format=report_format)

summary = report['executive_summary']
recommendations = "\n\n".join([
    f"**[{r['priority'].upper()}]** {r['title']}\n{r['recommendation']}"
    for r in report['recommendations'][:5]
])

return summary, recommendations, output_path

# Build UI
with gr.Blocks(title="Log Analyzer Agent") as app:
    gr.Markdown("# 🔍 Log Analyzer Agent")
    gr.Markdown("Upload a log file to analyze errors, patterns, and get recommendations")

    with gr.Row():
        with gr.Column():
            file_input = gr.File(label="Upload Log File", file_types=[".log"],
                                format_choice = gr.Dropdown(
                                    ["markdown", "html", "json", "text"],
                                    value="markdown",
                                    label="Report Format"
                                )
            analyze_btn = gr.Button("Analyze", variant="primary")

        with gr.Column():
            summary_output = gr.Markdown(label="Executive Summary")
            recommendations_output = gr.Markdown(label="Recommendations")
            report_path_output = gr.Textbox(label="Report Saved To")

```

```
analyze_btn.click(
    fn=analyze_log_file,
    inputs=[file_input, format_choice],
    outputs=[summary_output, recommendations_output, report_path_output]
)

if __name__ == "__main__":
    app.launch(share=False, server_port=7860)
```

```
bash

pip install gradio --break-system-packages
python ui/gradio_app.py
# Opens browser at http://localhost:7860
```

Option D — Streamlit Web UI

Create `ui/streamlit_app.py`:

```
python
```

```

import streamlit as st
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

from tools.log_parser import LogParser
from tools.error_detector import ErrorDetector
from tools.pattern_analyzer import PatternAnalyzer
from tools.report_generator import ReportGenerator
import tempfile

st.set_page_config(page_title="Log Analyzer Agent", layout="wide")
st.title("🔍 Log Analyzer Agent")

# Sidebar
with st.sidebar:
    st.header("Settings")
    report_format = st.selectbox("Report Format", ["markdown", "html", "json"])
    uploaded_file = st.file_uploader("Upload Log File", type=["log", "txt"])

# Main area
if uploaded_file:
    # Save uploaded file temporarily
    with tempfile.NamedTemporaryFile(delete=False, suffix='.log') as tmp:
        tmp.write(uploaded_file.read())
        tmp_path = tmp.name

    with st.spinner("Analyzing log file..."):
        # Parse
        parser = LogParser()
        parsed = parser.parse_file(tmp_path)

        col1, col2, col3, col4 = st.columns(4)

```

```
col1.metric("Total Lines", f"{parsed['total_lines']:,}")
col2.metric("Parsed Lines", f"{parsed['parsed_lines']:,}")
col3.metric("Format", parsed['format'].upper())
col4.metric("Parse Rate", f"{parsed['parsed_lines']/max(parsed['total_lines'], 1):.2%}")

# Detect errors
detector = ErrorDetector()
errors = detector.detect_errors(parsed)

col1, col2, col3 = st.columns(3)
col1.metric("Total Errors", errors['total_errors'])
col2.metric("Unique Types", errors['unique_error_types'])
col3.metric("Error Rate", f"{errors['error_percentage']:.2%}")

# Patterns
analyzer = PatternAnalyzer()
patterns = analyzer.analyze_patterns(parsed)

col1, col2 = st.columns(2)
col1.metric("Patterns Found", patterns['pattern_count'])
col2.metric("Anomalies", patterns['anomaly_count'])

# Generate report
generator = ReportGenerator()
results = {'log-parsing': parsed, 'error-detection': errors, 'pattern-analysis': patterns}
report = generator.generate_report(results)
output_path = generator.save_report(report, format=report_format)

# Display results
st.header("Executive Summary")
st.markdown(report['executive_summary'])
```

```

if report['recommendations']:
    st.header("Recommendations")
    for rec in report['recommendations'][:5]:
        color = {"urgent": "🔴", "high": "🟡", "medium": "🟢", "low": "🟠"}
        st.markdown(f"{color} **{rec['priority'].upper()}**: {rec['title']}")

st.success(f"Full report saved to: {output_path}")

# Download button
with open(output_path, 'r') as f:
    st.download_button("Download Report", f.read(), file_name=os.path.basename(output_path))
else:
    st.info("Upload a log file using the sidebar to begin analysis.")
    st.code("""
Supported formats:
- Apache / Nginx access logs
- Syslog (RFC3164 / RFC5424)
- JSON logs (JSONL)
- Application logs (timestamp + level + message)
- Docker container logs
- Kubernetes pod logs
""")

```

bash

```

pip install streamlit --break-system-packages
streamlit run ui/streamlit_app.py
# Opens browser at http://localhost:8501

```

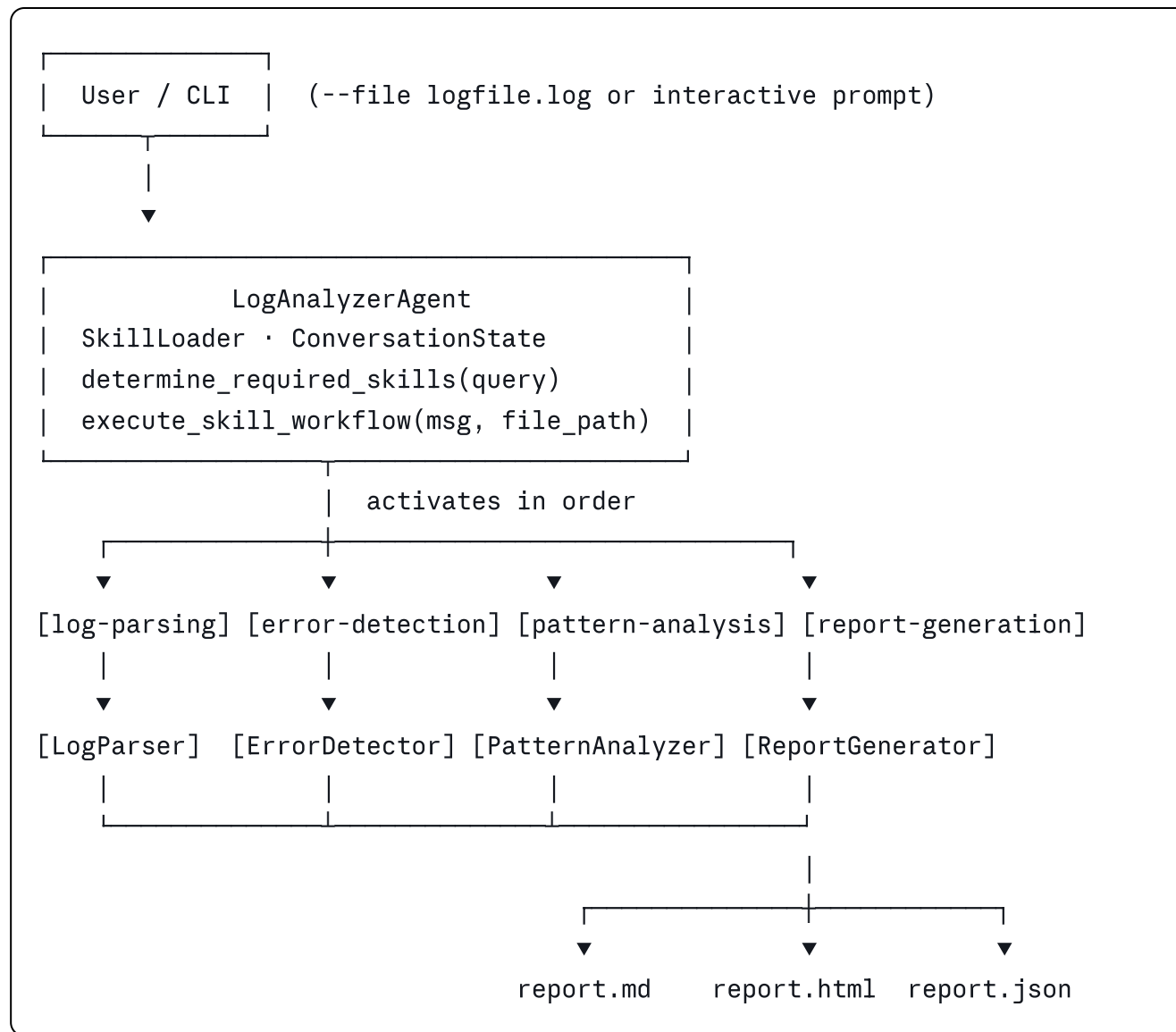
Comparison of Execution Options

Method	Interface	Setup	Best for
<code>python agent/main_agent.py --file</code>	Terminal	Instant	Automation, CI/CD, scripting
<code>python agent/main_agent.py --interactive</code>	Terminal chat	Instant	Exploration, iterative analysis
VS Code Python cells	Editor	Python extension	Development, debugging
Individual tools	Terminal	Instant	Custom pipelines, scripting
Gradio web app	Browser	<code>pip install gradio</code>	Demo, non-technical users
Streamlit web app	Browser	<code>pip install streamlit</code>	Dashboard, monitoring

Important note: This project is purely local — no LLM API calls happen during execution. The `anthropic` package is imported but not used in the current skills-based workflow. Skills invoke Python tools directly.

4. High-Level Architecture Diagram

See the diagram above in the guide. Here is the textual description:



5. Low-Level Component Diagram

See the diagram above in the guide. Full component breakdown:

agent/ layer

- `LogAnalyzerAgent` — main orchestrator, owns SkillLoader and ConversationState
- `SkillLoader` — reads and caches SKILL.md files, parses YAML frontmatter
- `ConversationState` — tracks active skills, stores results between steps

config/ layer

- `settings.py` — all thresholds, paths, error keywords, skill definitions

skills/ layer (4 skills, each with SKILL.md + scripts + references)

```
skills/
├─ log-parsing/
│   ├── SKILL.md           ← YAML frontmatter + 350-line instructions
│   ├── scripts/
│   │   ├── extract_fields.py
│   │   └─ normalize_timestamps.py
│   └─ references/
│       └─ log-formats.md   ← Full regex specs for all 8 formats
│
├─ error-detection/
│   ├── SKILL.md
│   ├── scripts/
│   │   └─ categorize_errors.py
│   └─ references/
│       └─ error-patterns.md ← Error catalog, stack trace processing
│
├─ pattern-analysis/
│   ├── SKILL.md
│   ├── scripts/
│   │   └─ find_patterns.py
│   └─ references/
```

```

|       └─ pattern-types.md ← 8 pattern types with confidence scoring
|
└─ report-generation/
    └─ SKILL.md
    └─ references/
        └─ report-templates.md ← Templates for all 4 output formats

```

tools/ layer (4 Python classes)

Class	Key methods	Input	Output
LogParser	detect_format(), parse_file(), _parse_apache(), _parse_json()	log file path	parsed_logs dict
ErrorDetector	detect_errors(), filter_errors(), categorize_errors(), group_similar_errors()	parsed_logs	error analysis dict
PatternAnalyzer	analyze_patterns(), find_recurring_patterns(), detect_anomalies(), analyze_trends()	parsed_logs	patterns dict
ReportGenerator	generate_report(), save_report(), _format_as_markdown(), _format_as_html()	all results dicts	saved report file

6. How to Generate and View the Report

Automatic Report (via Main Agent)

```
bash

python agent/main_agent.py --file tests/sample_data/application.log
# Report auto-saved to: outputs/log_analysis_report_YYYYMMDD_HHMMSS.md
```

Report Formats and How to View Them

Markdown Report

```
bash

# Generate
python agent/main_agent.py --file mylog.log
# → outputs/log_analysis_report_20260521_143052.md

# View in terminal
cat outputs/log_analysis_report_20260521_143052.md

# View with formatting (install glow)
brew install glow      # macOS
apt install glow       # Linux
glow outputs/log_analysis_report_20260521_143052.md

# View in VS Code
# Right-click the .md file → "Open Preview" (Ctrl+Shift+V)
```

HTML Report

```
bash

# Generate HTML format
python tools/report_generator.py /path/to/results/ --format html
# → outputs/log_analysis_report_20260521_143052.html

# View in browser
open outputs/log_analysis_report_20260521_143052.html      # macOS
xdg-open outputs/log_analysis_report_20260521_143052.html  # Linux
start outputs/log_analysis_report_20260521_143052.html     # Windows
```

JSON Report (for APIs and Dashboards)

```
bash

# Generate JSON format
python tools/report_generator.py /path/to/results/ --format json
# → outputs/log_analysis_report_20260521_143052.json

# Pretty print
cat outputs/log_analysis_report_20260521_143052.json | python -m json.tool

# Extract just executive summary
python -c "
import json
with open('outputs/log_analysis_report_20260521_143052.json') as f:
    r = json.load(f)
print(r['executive_summary'])
"
```

Report Structure Walkthrough

Every generated report follows this structure:

Log Analysis Report

Generated: 2026-05-21 14:30:52

Executive Summary ← Status, error rate, key findings (1 page)

Status: ⚠ WARNING

Error Rate: 2.3%

Critical Issues: 12

Key Findings: 3 bullet points

Priority Actions: 3 bullet points

Log Parsing Summary ← File info, parse success rate

Format: apache

Parse Rate: 97.8%

Time Range: 24 hours

Sample Entries: 3 examples

Error Analysis ← Categorized, grouped, ranked

Total Errors: 1,523

Error Distribution: bar chart (ASCII)

Top 10 Errors: ranked by frequency

Critical Errors: with stack traces

Pattern Analysis ← Patterns, anomalies, trends

Patterns Found: 12

```
Anomalies: 5
Trends: error rate increasing +45%
Peak Times: 14:00 (5x average)
```

Visualizations

```
Error Timeline:  ASCII bar chart by hour
Error Distribution: ASCII horizontal bars
```

Recommendations ← Prioritized action items

- URGENT: Fix database connection pool
- HIGH: Investigate memory leak
- MEDIUM: Optimize cache strategy

Appendix

```
Sample log entries
Methodology notes
```

Sample Use Case: End-to-End Walkthrough

Scenario: Your production app has been slow. You downloaded logs from the server.

```
bash
```

```
# Step 1: Copy logs from server
scp prod-server:/var/log/app/application.log ./prod_logs.log

# Step 2: Run full analysis
python agent/main_agent.py --file prod_logs.log

# Step 3: Console output (happens in ~5 seconds)
# ✓ Parsed 156,234 log entries
# ✓ Found 1,523 errors (47 unique types)
# ✓ Identified 12 patterns, 5 anomalies
# ✓ Report saved to: outputs/log_analysis_report_20260521_143052.md

# Step 4: Read executive summary
head -50 outputs/log_analysis_report_20260521_143052.md

# Step 5: Open HTML report in browser for full view
open outputs/log_analysis_report_20260521_143052.html

# Step 6: Share JSON with your team's monitoring system
curl -X POST https://your-dashboard.com/api/import \
  -H "Content-Type: application/json" \
  -d @outputs/log_analysis_report_20260521_143052.json
```

What the agent found:

- ⚠ WARNING status (2.3% error rate, trending up)
- 12 critical errors: database connection pool exhaustion
- Pattern: DB errors recurring every 30 minutes (correlates with scheduled job)
- Anomaly: Traffic spike at 14:23 (15x normal — likely external event)
- Recommendation: Increase connection pool from 50 to 100 connections

Adding Custom Report Sections

To add custom sections to your report, edit `tools/report_generator.py`:

```
python
```

```
def create_detailed_sections(self, data: Dict[str, Any]) -> Dict[str, str]:
    sections = {}
    sections['parsing'] = self._create_parsing_section(data.get('parsing', {}))
    sections['errors'] = self._create_error_section(data.get('errors', {}))
    sections['patterns'] = self._create_pattern_section(data.get('patterns', {}))

    # Add your custom section:
    sections['security'] = self._create_security_section(data)

    return sections

def _create_security_section(self, data: Dict[str, Any]) -> str:
    """Custom: highlight auth failures and suspicious IPs"""
    errors = data.get('errors', {})
    auth_errors = [
        e for e in errors.get('grouped_errors', [])
        if e['category'] == 'authentication'
    ]

    section = "## Security Analysis\n\n"
    section += f"**Authentication Failures**: {sum(e['occurrences'] for e in auth_errors)}\n\n"

    for error in auth_errors[:5]:
        section += f"- {error['message_template']} ({error['occurrences']} times)\n\n"

    return section
```

Quick Reference Card

EXECUTE:

Full analysis: `python agent/main_agent.py --file mylog.log`
Interactive: `python agent/main_agent.py --interactive`
Parse only: `python tools/log_parser.py mylog.log`
Errors only: `python tools/error_detector.py parsed.json`
Patterns only: `python tools/pattern_analyzer.py parsed.json`

SKILL TRIGGERS:

"parse / .log" → log-parsing
"error / failure / detect" → error-detection
"pattern / anomaly / trend" → pattern-analysis
"report / summary" → all 4 skills

REPORTS:

Saved to: `outputs/log_analysis_report_YYYYMMDD_HHMMSS.{ext}`
Formats: `.md (default) | .html | .json | .txt`
View MD: `cat / glow / VS Code preview`
View HTML: `open in any browser`
View JSON: `python -m json.tool | jq`

WEB UI:

Gradio: `pip install gradio && python ui/gradio_app.py`
Streamlit: `pip install streamlit && streamlit run ui/streamlit_app.py`

TESTS:

All tests: `bash run_tests.sh`
Parser: `python tests/test_log_parser.py`
Detector: `python tests/test_error_detector.py`

Log Analyzer Agent — Skills-based architecture following Claude Skills Handbook best practices